Module 2: Reading and Understanding Datasets

Part 1: Reading the Dataset

In this module, we will work with the Multi-Hospital Lab Results dataset.

The dataset contains laboratory records collected from three hospitals.

We will use Pandas to read and explore the dataset before performing any data cleaning.

Dataset source: Kaggle - Multi-Hospital Lab Results (Messy Data)

https://www.kaggle.com/datasets/nudratabbas/multi-hospital-lab-results-messy-data

In [5]:
```
# Example Dataset one
import pandas as pd

# Read the dataset
df = pd.read_csv("Dataset/Hospital_lab/multi_hospital_lab_results.csv")

# Display first five rows
display(df.head())
```

	hospital	patient_id	test_name	test_value	unit	reference_range	collection_date
0	Hospital_A	1966	Fasting Glucose	8.56	mmol/L	70-99	2025-08-15
1	Hospital_C	1826	Glucose	-999	mmol/L	70-99	2024-07-04
2	Hospital_A	1606	Cholesterol	6.45	mmol/L	125-200	2024-11-13
3	Hospital_B	1277	Fasting Glucose	-999	mmol/L	3.9-5.5	2024-04-29
4	Hospital_A	1841	Serum Cholesterol	5.61	mmol/L	<200	2024-08-05

Understanding Rows and Columns

A DataFrame organizes information into rows and columns.

- **Rows:** Individual laboratory records.
- **Columns:** Attributes describing each record.
- **Index:** Row labels used by Pandas.

Each row represents one laboratory test record. A patient may appear in multiple records.

Part 2: Understanding Dataset Structure

Before processing a dataset, we need to understand its size, columns, and data types.

We will use:

- `shape` - Number of rows and columns
- `columns` - Column names
- `dtypes` - Data types
- `info()` - General dataset information

In [13]:
```
# Number of rows and columns
print("Dataset shape:", df.shape)

# Column names
print("\nColumn names:")
print(df.columns.tolist())

# Data types
print("\nData types:")
print(df.dtypes)

# General information
df.info()
```

```
Dataset shape: (1500, 7)

Column names:
['hospital', 'patient_id', 'test_name', 'test_value', 'unit', 'reference_range',
'collection_date']

Data types:
hospital             str
patient_id         int64
test_name            str
test_value           str
unit                 str
reference_range      str
collection_date      str
dtype: object
<class 'pandas.DataFrame'>
RangeIndex: 1500 entries, 0 to 1499
Data columns (total 7 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   hospital         1500 non-null   str
 1   patient_id       1500 non-null   int64
 2   test_name        1500 non-null   str
 3   test_value       1461 non-null   str
 4   unit             1500 non-null   str
 5   reference_range  1500 non-null   str
 6   collection_date  1500 non-null   str
dtypes: int64(1), str(6)
memory usage: 82.2 KB
```

Understanding Data Types

Pandas identifies the data type of each column.

In our dataset:

- `patient_id` is stored as an integer.
- `test_value` is stored as a string.
- `collection_date` is stored as a string.
- Other descriptive columns contain text.

Data types are important because they determine which operations we can perform on the data.

We will investigate incorrect data types in Module 3.

3.1 Displaying Dataset Records

We use:

- `head()` to display the first rows.
- `tail()` to display the last rows.
- `sample()` to display randomly selected rows.

These methods help us inspect the dataset without modifying its contents.

3.2 Exploring Hospital Records

We will examine how laboratory records are distributed across hospitals.

We use:

- `unique()` to identify hospital names.
- `nunique()` to count different hospitals.
- `value_counts()` to count records per hospital.

In [11]:
```
missing_values = df.isnull().sum()

print(missing_values)

#checking for duplicates
duplicates = df.duplicated().sum()

print(duplicates)
```

```
hospital            0
patient_id          0
test_name           0
test_value         39
unit                0
reference_range     0
collection_date     0
dtype: int64
0
```

Exploring Dataset Columns

We will explore the hospital lab dataset to understand the values stored in different columns.

We will use:

- `unique()` to display unique values.
- `nunique()` to count unique values.
- `value_counts()` to count repeated values.

In [7]:
```
# Unique hospitals
print(df["hospital"].unique())

# Number of hospitals
print(df["hospital"].nunique())

# Number of records per hospital
print(df["hospital"].value_counts())
```

```
<StringArray>
['Hospital_A', 'Hospital_C', 'Hospital_B']
Length: 3, dtype: str
3
hospital
Hospital_C    519
Hospital_A    502
Hospital_B    479
Name: count, dtype: int64
```

3.3 Exploring Laboratory Test Names

The `test_name` column describes the laboratory test associated with each record.

We will identify:

- Different test names.
- Number of unique test names.
- Frequency of each test name.

This helps us understand the types of laboratory measurements represented in the dataset.

In [18]:
```
# Unique test names
print(df["test_name"].unique())

# Number of unique test names
print(df["test_name"].nunique())

# Frequency of each test name
print(df["test_name"].value_counts())
```

```
<StringArray>
[    'Fasting Glucose',             'Glucose',         'Cholesterol',
   'Serum Cholesterol', 'Glycated Hemoglobin',      'Hemoglobin A1c',
         'Blood Sugar',          'Total Chol',               'HbA1c']
Length: 9, dtype: str
9
test_name
Cholesterol            181
Serum Cholesterol      178
Hemoglobin A1c         175
HbA1c                  172
Blood Sugar            167
Glycated Hemoglobin    164
Fasting Glucose        161
Glucose                155
Total Chol             147
Name: count, dtype: int64
```

3.4 Exploring Measurement Units

Laboratory measurements may use different units.

The `unit` column stores the measurement unit associated with each laboratory result.

We will inspect the available units and their frequencies.

In [19]:
```
# Unique measurement units
print(df["unit"].unique())

# Frequency of units
print(df["unit"].value_counts())

# Display test names, values, and units
display(df[["test_name", "test_value", "unit"]].head(10))
```

```
<StringArray>
['mmol/L', '%', 'mg/dL']
Length: 3, dtype: str
unit
%         511
mmol/L    506
mg/dL     483
Name: count, dtype: int64
```

	test_name	test_value	unit
0	Fasting Glucose	8.56	mmol/L
1	Glucose	-999	mmol/L
2	Cholesterol	6.45	mmol/L
3	Fasting Glucose	-999	mmol/L
4	Serum Cholesterol	5.61	mmol/L
5	Glycated Hemoglobin	7.0	%
6	Hemoglobin A1c	5.2	%
7	Glycated Hemoglobin	6.9	%
8	Blood Sugar	80.77415407811968	mg/dL
9	Total Chol	208.94318817997245	mg/dL

3.5 Exploring Patients and Collection Dates

The dataset includes patient identifiers and sample collection dates.

We will check:

- Number of unique patient identifiers.
- Number of laboratory records.
- Examples of collection dates.

Patient identifiers are used to distinguish patients within the dataset.

In [20]:
```
# Number of unique patient IDs
print("Unique patient IDs:", df["patient_id"].nunique())

# Total laboratory records
print("Total records:", len(df))

# First ten collection dates
print(df["collection_date"].head(10))
```

```
Unique patient IDs: 760
Total records: 1500
0    2025-08-15
1    2024-07-04
2    2024-11-13
3    2024-04-29
4    2024-08-05
5    2024-12-23
6    2025-05-22
7    2024-08-30
8    2026-01-21
9    2025-09-08
Name: collection_date, dtype: str
```

3.6 Descriptive Summary

The `describe()` method summarizes dataset columns.

For numerical columns, it provides statistics such as minimum, maximum, and average.

For text columns, it provides information such as unique values and the most frequent value.

We will use it to understand the dataset before data cleaning.

In [21]:
```
# Numerical summary
display(df.describe())

# Summary of all columns
display(df.describe(include="all").T)
```

	patient_id
count	1500.000000
mean	1503.090667
std	288.548052
min	1000.000000
25%	1263.500000
50%	1504.500000
75%	1741.000000
max	2000.000000

	count	unique	top	freq	mean	std	min	25%
hospital	1500	3	Hospital_C	519	NaN	NaN	NaN	NaN
patient_id	1500.0	NaN	NaN	NaN	1503.090667	288.548052	1000.0	1263.
test_name	1500	9	Cholesterol	181	NaN	NaN	NaN	NaN
test_value	1461	800	-999	36	NaN	NaN	NaN	NaN
unit	1500	3	%	511	NaN	NaN	NaN	NaN
reference_range	1500	9	<5.7%	194	NaN	NaN	NaN	NaN
collection_date	1500	646	2025-07-02	7	NaN	NaN	NaN	NaN

Part 4: Initial Data Quality Observations

Before cleaning a dataset, we should inspect its completeness and consistency.

We will check:

- Missing values
- Duplicate rows
- Column data types

Our goal is to observe possible issues, not to correct them yet.

In [22]:
```
# Missing values
print("Missing values:")
print(df.isna().sum())

# Duplicate rows
print("\nDuplicate rows:")
print(df.duplicated().sum())

# Data types
print("\nData types:")
print(df.dtypes)
```

```
Missing values:
hospital            0
patient_id          0
test_name           0
test_value         39
unit                0
reference_range     0
collection_date     0
dtype: int64

Duplicate rows:
0

Data types:
hospital             str
patient_id         int64
test_name            str
test_value           str
unit                 str
reference_range      str
collection_date      str
dtype: object
```

Practical Exercises: Hospital Lab Dataset

Using the Multi-Hospital Lab Results dataset, complete the following tasks:

1. Display the first 10 laboratory records.
2. Find the number of rows and columns.
3. Display all column names.
4. Identify the data type of each column.
5. Find the number of records from each hospital.
6. Identify all unique laboratory test names.
7. Find the number of unique measurement units.

8. Count missing values in each column.
9. Count duplicate rows.
10. Write three observations about the dataset.

Do not modify the original dataset.

Module 2 Summary

In this module, we learned how to:

- Read a CSV dataset using Pandas.
- Understand DataFrame rows and columns.
- Check dataset dimensions and data types.
- Explore categorical values.
- Summarize dataset contents.
- Make initial observations about data quality.

Next Module: Identifying Data Quality Issues

We will investigate missing values, duplicates, incorrect data types, inconsistent text, invalid measurements, and date-format problems.

We will use the same hospital lab dataset.